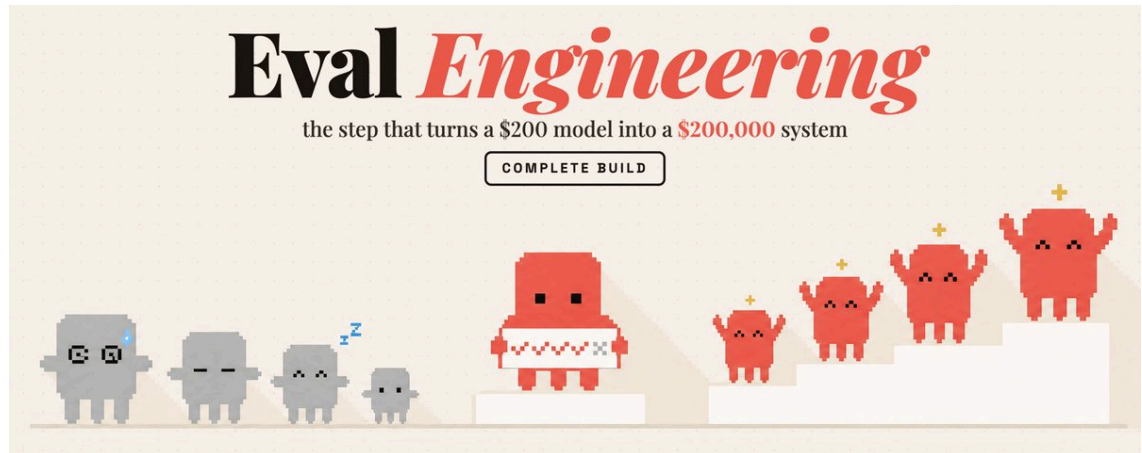




← Post



Argona ✓
@Argona0x



Eval Engineering: the step that turns a \$200 model into a \$200,000 system (complete build)

Everyone is renting the same brain now.

The model you can have for a couple of hundred a month is, give or take, the model a company with a thousand engineers is running. That was supposed to level everything. Instead it made the gap between two teams renting the identical brain wider than the gap between two models.

A travel agent, the software kind, answered a question about a trip: an exchange rate to one decimal place, the temperature for the week, the opening hours of a museum. Specific, clean, useful-sounding. **All of it invented.**

The search tool had come back empty, and the model quietly filled the hole and handed its own invention over as a fact it had looked up.

Then the team measured it across 100 real sessions and got two numbers that should never sit that far apart.

How good the answers were: **83.9%.**

How much of those answers was grounded in what the tools actually returned: **32.3%.**

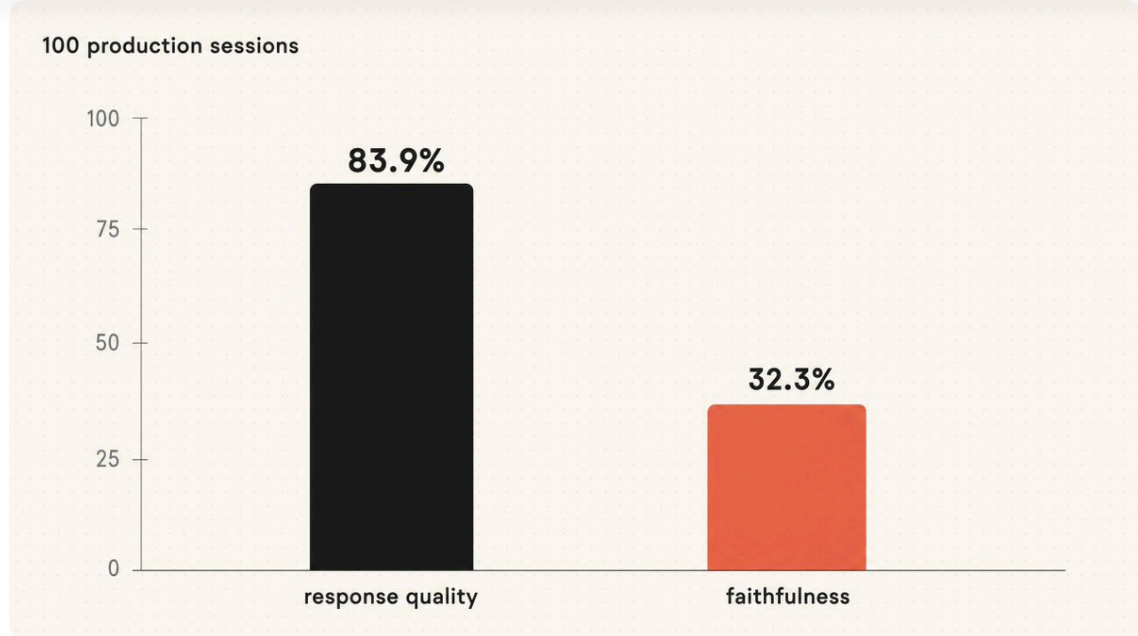
The agent wrote beautifully and told the truth about a third of the time.

Don't miss what's happening

People on X are the first to know.

Log in

Sign up



83.9% response quality against 32.3% faithfulness, one and the same run

Same agent, same day: the left bar is what the demo shows, the right bar is what a customer actually gets.

A better model would not have caught this. What was missing was the layer that decides whether an answer was right, and then does something with the verdict.

That layer is the whole distance between an AI that impresses people and an AI that gets paid. It is the cheapest line in the stack and **the only one nobody can sell you**, because it encodes your own definition of correct.

This one is not a skim. Open a terminal beside it. The setup starts with a single command you already own, and it ends somewhere that sounds unreasonable: pull requests that merge with nobody reading them.

Before we get into it, follow me on X and join my Telegram channel where post more AI content every day. Both are free.

X - <https://x.com/Argona0x>

Telegram - <https://t.me/+r0cll4-MMC03ZjAy>

What eval engineering is

A company running agents in production said the quiet part in one sentence. Across successive versions of their agent they changed **no model, no prompts**,

Don't miss what's happening

People on X are the first to know.

Log in

Sign up

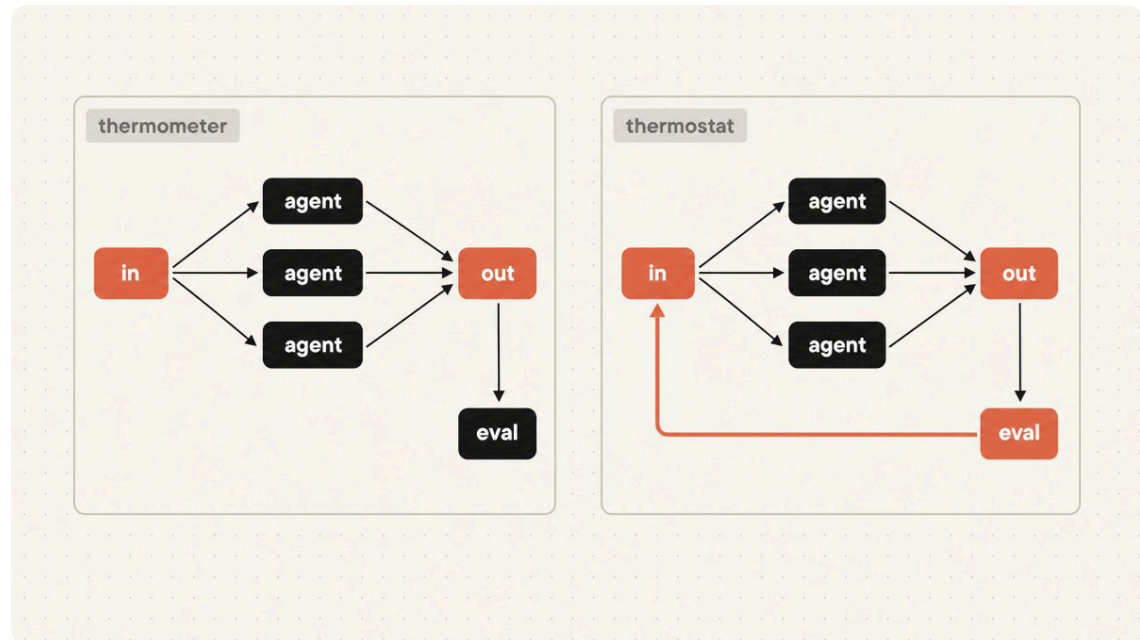
Same brain. Different examiner. Better product.

Their own conclusion afterwards was blunter: the evals should have been in the system on day one, not month nine.

A thermometer tells you the room is cold. A thermostat turns the heat on.

Almost everyone building with AI owns a thermometer at best: a dashboard, a feeling, a Friday afternoon where somebody scrolls outputs and says it seems worse than last week.

Eval engineering is the wiring that runs from the reading to the furnace.



← thermometer against thermostat, with the verdict returning into the graph

The whole difference is the return arrow on the right: the verdict goes back into the graph and changes what runs next.

This arrived in a fixed order. One agent in a loop came first, so it could try, look at what came back, and try again. Then many agents laid out as a graph, so work that never depended on each other ran side by side instead of standing in line.

Loops, then graphs, then evals.

<https://x.com/i/web/status/2080626046903157126>

The graph was the last upgrade that made you faster. The judge decides whether

Twenty agents running at once, all reporting to one frozen judge, is twenty times as many places for a wrong answer to look finished.

It is also the only part of the system that gets more valuable every week you run it. The model is a rental. The examiner is yours, and every failure you feed it stays there permanently.

Which makes the price of entry strange. The examiner costs almost nothing to install, and the model it examines costs about \$200 a month.

Step 1 • The eval engine you already own

You assume this starts with a platform: a contract, a per-seat price, a quarter of setup before the first useful number.

That assumption is why most people never build the layer at all, and it stopped being true about four weeks ago.

The blind test

1. Two researchers ran the honest test. They took 100 real production traces from a live voice agent, had a human expert mark every failure by hand, built a taxonomy of **39 labeled failures**, then hid the labels and handed the same pile to every evaluation system on the market.

Find them yourself.

The interesting row was not the winner:

1. **Braintrust Loop:** 87.2% of the human-flagged failures recovered.
2. **Codex on GPT-5.5 High:** 84.6% recall, 82.8% precision.
3. **LangSmith:** 79.5%.
4. **Arize AX:** 74.4% recall, and the cleanest precision of the group at 91.0%.

A general coding agent, on a subscription you already pay for, landed above two dedicated evaluation platforms. One of them wrote the conclusion plainly: you can get similar results from using your coding agent.

Then the platforms did something stranger than losing. They moved in.

Inside four weeks, four evaluation vendors shipped their expertise as a **<skill installed into somebody else's coding agent:** LangChain on 22 July, Galileo, AWS under Apache 2.0, and Arize putting its trace extraction into skills aimed at, in their words, your favorite coding agent.

It is a category unbundling itself into the terminal you already have open.

Installing the examiner

Installing all of it takes three lines.

python



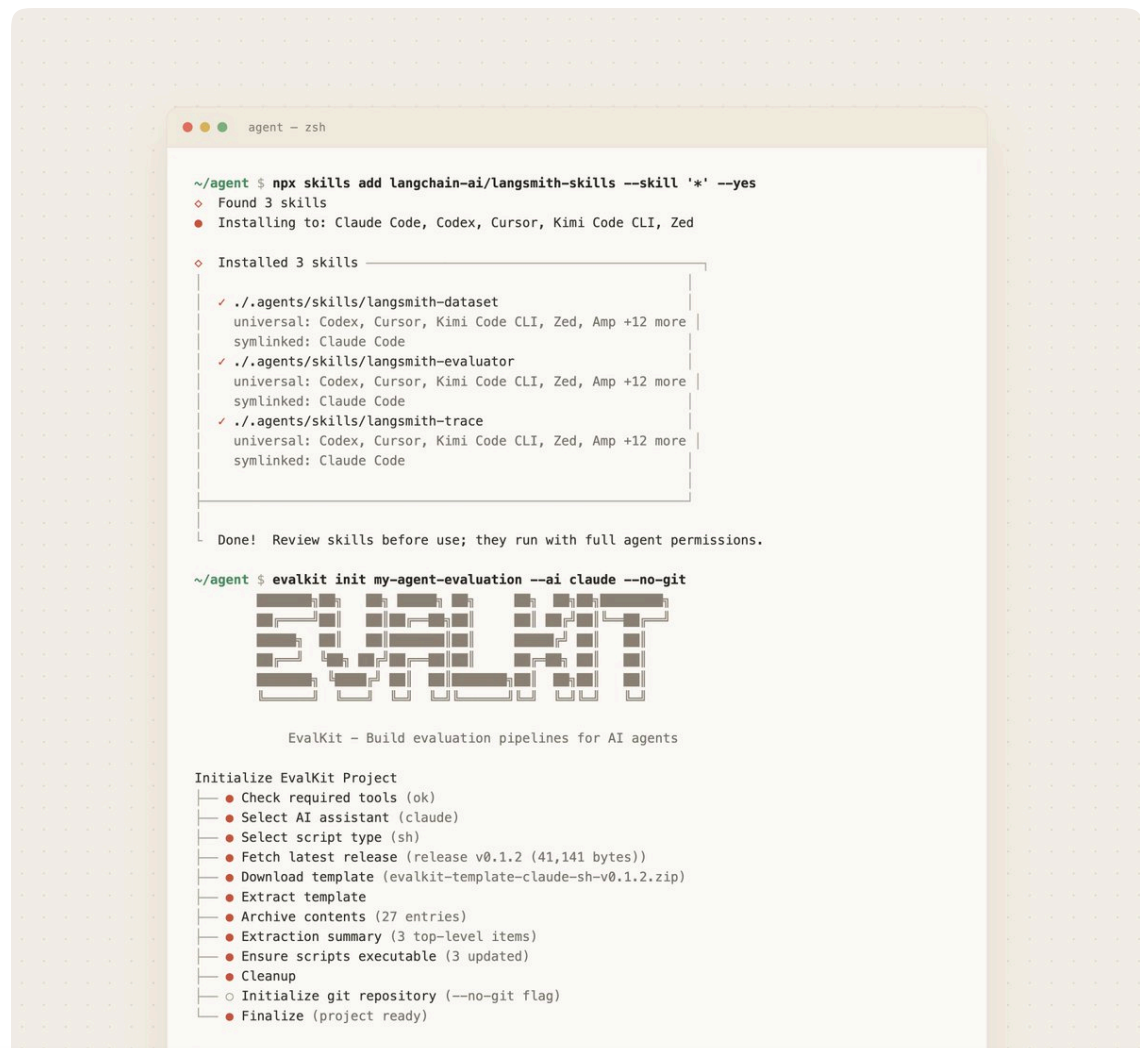
```
npm skills add langchain-ai/langchain-skills --skill '*' --yes --
npm skills add langchain-ai/langsmith-skills --skill '*' --yes --
uv tool install evalkit --from git+https://github.com/aws-labs/Agent
```

Inside Claude Code the same thing installs as a plugin:

python



```
/plugin marketplace add langchain-ai/langchain-skills
/plugin install langchain-skills@langchain-skills
```



Don't miss what's happening

People on X are the first to know.

Log in

Sign up

A real run on a real machine: the skills land as global files, and **evalkit init** lays out the folder your evaluation will live in.

Two details in there are worth more than the commands.

1. Two repositories, two jobs: **langchain-skills** builds the evals, **langsmith-skills** pulls the real production runs to build them from. Almost everyone installs the first, then wonders where the material comes from.
2. Not a Claude Code feature: those skills follow the open skills specification, so the identical files load into Codex, Cursor, Windsurf and Goose.

The AWS kit runs on six commands, each a phase writing into an **eval/** folder the next one reads:

python



```
evalkit init my-agent-evaluation
# then, inside your coding agent:
/evalkit.plan Evaluate my agent at ./my_agent for grounding and to
/evalkit.data
/evalkit.trace
/evalkit.run_agent
/evalkit.eval
/evalkit.report
```

The last command is the one worth the whole setup. **/evalkit.report** hands back prioritized recommendations pointing at specific locations in your code.

The examiner is installed. Now it has to be allowed to do something.

Step 2 • Make the score change the next edge

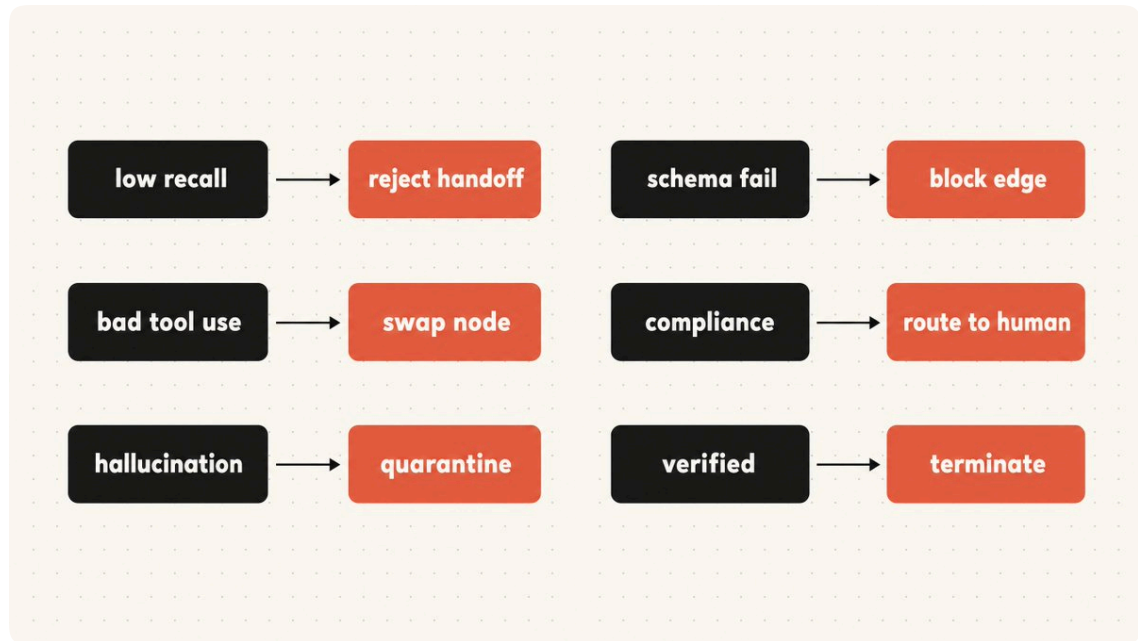
You will build your first evaluation, get a number, look at it, and feel nothing happen.

That feeling is correct. A number in a report has no path back to the run it measured.

One line fixes the whole discipline, and it belongs to a 21-year-old founder in Tokyo with 258 followers whose post on it got two likes:

A score that never changes behavior is analytics. An eval that changes the next edge is engineering.

low context recall → reject the handoff
 bad tool use → retry **or** swap the node
 hallucination → quarantine the branch
 schema failure → block the edge
 compliance risk → route to human review
 verified completion → terminate the run



← six verdicts, six structural actions on the run in progress

Every one of those six is a routing decision the graph executes. The eval steers the run mid-flight, one edge at a time.

Rules for the judge itself

The examiner needs hygiene of its own, and almost nobody sets this part up correctly.

1. Judge from another family: a model recognizes its own writing and grades it kinder once it does. One evaluation team runs Sonnet as the judge for output generated by Haiku, on purpose. One line on X gets it in fewer words: same family generates and grades, so the blind spots are shared.
2. Write the rubric as one line: the working form is literally ***Pass iff [the independently observable successful outcome]***. One primary verdict, never a bundle of proxy scores.
3. Split the work by kind: the judge decides the semantic calls, plain code decides the objective ones. Did the test pass, does the file exist, did the state change.
4. Never reward the shape of an answer: the rule written into the skill forbids

5. Pin the judge and log its version: an examiner that silently upgrades makes every score before and after incomparable, and you will not notice for a month. The version pin is the one people skip, and the one that makes a month of scores unreadable after the fact.

Optimize against a judge long enough and the agent learns to look right rather than be right.

Now the score has somewhere to go. The next problem is where good tests come from, because inventing them at a desk is why everybody's first set is useless.

Step 3 • Turn a failed run into an eval

Tests you invent from imagination protect you from failures you already imagined.

The ones that cost money are sitting in your logs right now, wearing a timestamp.

The whole loop is five steps long:

python



```
mine traces -> identify a failure -> build an eval -> improve the
```

Where the tests come from

The first step carries all the weight. The skill that does this professionally names which runs to pull, and it starts with **25 complete traces, no more**, chosen so good and bad behavior sit next to each other:

1. A normal request that finished: your baseline for what working looks like.
2. A request the user confirmed: the rare trace where you know the answer was good.
3. A request the user corrected or rephrased: the correction is the label, free of charge.
4. A run with a failed, empty or repeated tool call: repetition means a loop, emptiness means an invented answer is coming.
5. A run with an external failure: a timeout or a rate limit, where the only thing tested is how your agent behaves when the world says no.

Each one gets written up in four lines, and this template is the part worth stealing:

python



Attribution: agent behavior, dependency behavior, or unclear
Eval candidate: the capability to preserve or improve

Attribution is where beginners lose a week.

The same lookup called twice with identical arguments is a loop in your agent. A 429 coming back is somebody else's limit, and it only becomes your eval if your agent was supposed to recover from it.

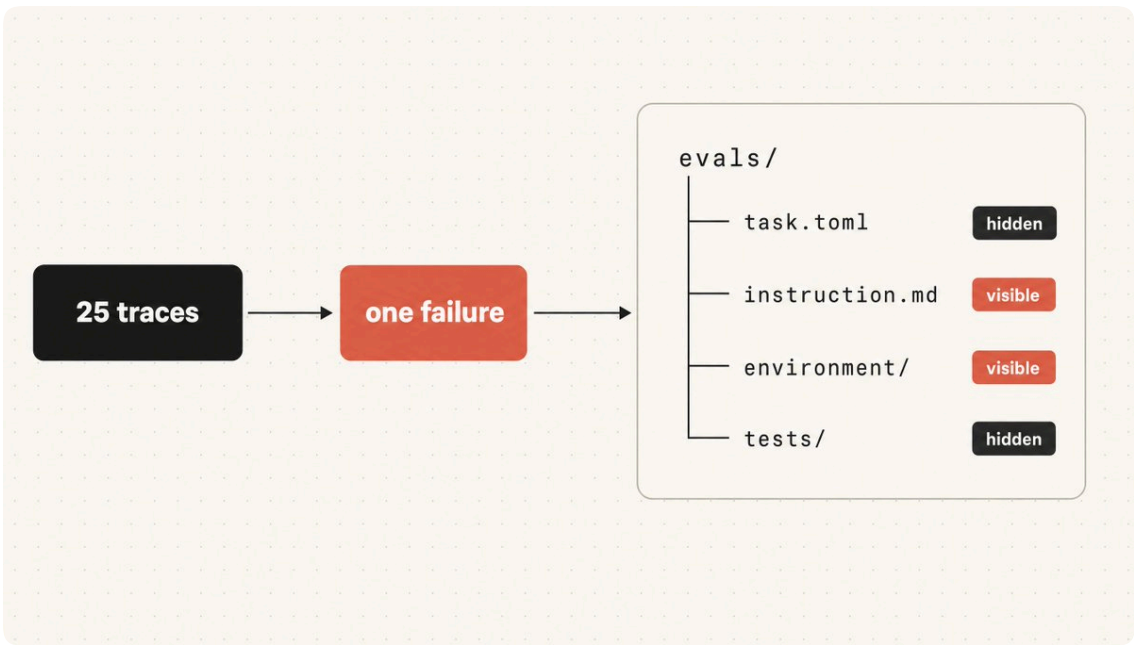
From finding to folder

Then the finding becomes a folder, and the folder is the format the tooling runs:

python



```
evals/<task-id>/
├─ task.toml
├─ instruction.md
├─ environment/
└─ tests/
```



← 25 traces → one failure → a Harbor task folder: what the agent sees, what stays hidden

One capability per folder. The instruction and the environment are visible to the agent being tested. The expected outcome, the rubric and the judge's credentials are not, and that separation is the only reason the score means anything.

Don't miss what's happening

People on X are the first to know.

Log in

Sign up

1. Never treat the recorded answer as truth: the trace tells you what your agent did, never what it should have done. Take the answer key from tests, source records, policy, known state or a person.
2. Test the test before you trust it: hand the verifier two fake results by hand, one clearly correct and one plausible but wrong. If either goes the wrong way, the rubric is broken, not the agent.
3. Watch for the environment giving away the answer: if the setup hands over the result before the agent reaches the tool it was supposed to use, the task passes forever and measures nothing.

Anything that costs money or writes to production gets simulated rather than called, so the suite runs as often as you like without a bill.

One instruction starts all of it, in the agent you already have open:

markdown



Use the eval-engineering skill.

Map this repository's agent: entrypoint, tools, backing data, and result looks like. Then read the 25 traces in ./traces and propose eval candidates grounded in what actually failed there.

Recommend one. Do not implement until I choose.

Build it as a Harbor task under evals/, keep the rubric and expected hidden from the target, and test the verifier on one passing and one wrong result before the real run.

The interview step is deliberate. The team that wrote the skill found that questioning the user beats one-shot generation every time, for the plain reason that the definition of correct lives in your head and nowhere in the model.

Run it a handful of times and every failure stops being an incident and becomes a permanent test.

Step 4 • Let the graph merge its own work

Every pull request an agent opens lands in the same place: a human queue.

You become the bottleneck of your own automation, and the fleet you built runs at the speed of one tired reviewer.

The way out looks nothing like trusting the model more. When an agent opens a

Don't miss what's happening

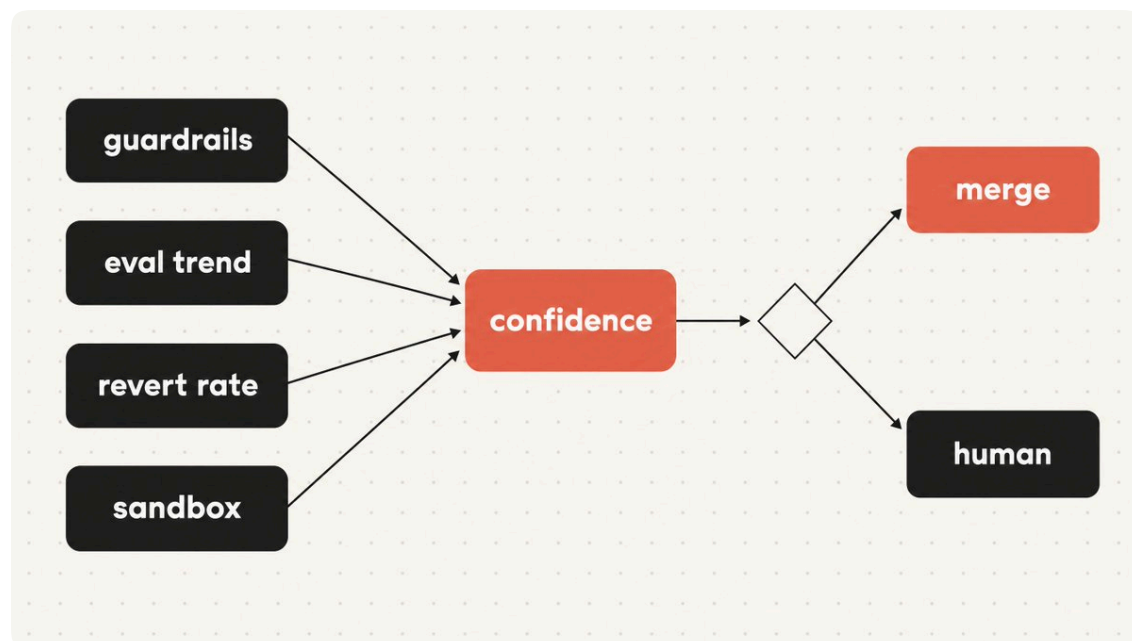
People on X are the first to know.

Log in

Sign up

1. **Guardrails result:** a deterministic pass or fail on the blocking standards. No model involved.
2. **Recent eval trajectory:** how this exact version of this agent has been scoring lately.
3. **Historical revert rate:** how often this agent, on this repository, on this class of change, has had its work rolled back before.
4. **Sandbox outcome:** did it run.

bove the threshold it merges itself. Below it, a human gets it **with the failing signal named**, so the review starts at the problem instead of at line one.



← four signals → one confidence score → merge itself, or route to a human

Three of those four are history and deterministic checks, and exactly one of them touches the model at all.

Trust in an agent is an actuarial calculation.

What you are building is a track record with a price on it, the same way an insurer builds one, and it gets sharper every week whether or not the models improve.

What it looks like when it runs

At the same company that changed nothing but its evals, **19 of every 20 pull requests** on the fully autonomous agent merge with no human involved.

Across the top agents, about three quarters of merged work goes in without a single human edit, the revert rate stays in the low single digits, and Guardrails alone bounces one pull request in five before a person ever sees it.

In the past 90 days, I've approved and merged around 1,500 pull requests. I haven't looked at a line of code. How can I put so much trust in agents? I don't trust them at all. I also don't trust my ability to code review their work. But I do trust my ability to constrain their work.

The constraint is the product.

And the warning worth more than the formula comes from a team that ran 285 iterations of a self-improving codebase and came out with 1,094 merged pull requests and zero regressions

Their own line is the one to keep: **38 green tests coexisted with a completely broken product.**

A suite can go all green while the product it guards falls apart, which is why the loop has to converge on the spec rather than on the score.

Turn it on the careful way:

1. Run it in shadow first: the gate scores every pull request and merges none of them, for at least 4 hours of real traffic.
2. Set a deviation threshold of 2%: if the automated verdict and the human verdict disagree more than that, the gate stays closed.
3. Sample traces at 1% to 5%: full capture on everything is a cost you do not need to carry.

A two-person shop where every agent-written change waits on one reviewer can take on about as much work as that reviewer can read.

With the gate closed on the risky slice and open on the boring 80%, the same two people start quoting on the contracts they used to decline.

The model bill does not move. Everything else does.

Step 5 • The evals to build this week

A suite that is too slow or too vague never gets run, so this is the part to save.

Start with three measurements, not twelve. The three that exposed the travel agent are a working default for any agent that calls tools:

1. **Faithfulness:** is the answer grounded in what the tools actually returned. This is the one that sat at 32.3% while every other number on the dashboard looked fine.
2. **Tool parameter accuracy:** right tool right arguments

If your agent ships code, score the change instead. The five dimensions used in production on every pull request: intent and decision, execution and artifact, completeness and usefulness, instruction and boundary, efficiency.

Pick the dataset type on purpose. Four exist: ***final_response*** for the answer alone, ***single_step*** for one decision in isolation, ***trajectory*** for the whole path the agent took, and ***RAG*** for retrieval quality.

Grading only the final response is how an agent reaches a correct answer through a broken sequence with nobody noticing.

Size it so it stays alive. Hold out 300 to 800 cases, and keep 500 of them running **in under 5 minutes.**

A suite that takes longer than a coffee break stops being run.

The first five evals, in order:

1. **Empty tool result:** the tool returns nothing, and the agent has to say so instead of inventing the number.
2. **Repeated call:** the same lookup with the same arguments twice. That is a loop, and the eval fails it.
3. **Boundary refusal:** asked for something outside its permissions, the agent declines cleanly instead of hunting for a route around.
4. **Handoff integrity:** what the previous node produced is what the next node reads, with nothing invented in between.
5. **Verified completion:** done means a real signal says done, never the agent's own word for it.

this week

3 measurements

- faithfulness
- tool accuracy
- response quality

1 dataset type

- trajectory

5 evals

- empty result
- repeated call
- boundary refusal
- handoff integrity
- verified completion

← this week: three measurements, one dataset type, five evals (the save-card)

Three measurements, one dataset type, five evals. That is an afternoon of work, and every week after it the suite is worth more than it was the week before.

The people who go first

The model was never the interesting part. It is a rental, identical for everyone, and it will be replaced twice before the end of the year.

Don't miss what's happening

People on X are the first to know.

[Log in](#)

[Sign up](#)

That is what decides whether the **\$200** on your card statement produces a demo or produces a business.

Most people will go back to scrolling outputs on a Friday and deciding it feels about right.

The ones who go first spend one afternoon wiring the thermostat, and then spend the next year with an agent that cannot break the same thing twice.

Three lines hold the whole discipline:

- 1. Measure the path the agent took, never only the answer it landed on.
- 2. A verdict that does not change the next edge is a report.
- 3. Any failure you do not turn into a permanent test, you will meet again.

Install one skill. Pull 25 traces. Build one eval today, and add one every time something breaks.

If you want to stay up to date with everything happening in AI, follow me on X and Telegram:

X - <https://x.com/Argona0x>

Telegram - <https://t.me/+r0cll4-MMC03ZjAy>

3:32 PM · Jul 28, 2026 · **9.8K** Views

3

47

80

Archive

@ArchiveExplorer · 8h

loops then graphs then evals is the cleanest way ive seen anyone order it

the graph made me fast

1

1

88

Fiseo

@0xFiseo · 56m

The \$200,000 is not the model. It is the harness, the loop, and the graph.

Eval engineering is the discipline of building everything except the model.

14

Danko

@DankoWeb3 · 3h

So interesting stuff bros

Don't miss what's happening

People on X are the first to know.

[Log in](#)

[Sign up](#)